プログラミング言語実験-第 5 回-

2412747　江尾祐哉

1 問目

・コンパイルおよび実行方法

「g++ 2412747_kadai5-1.cpp -o 2412747_kadai5-1 $(pkg-config --cflags --libs opencv4)」とコンパイルし、「./2412747_kadai5-1」と実行する。

・実装概要

この課題では、前回のように行列のクラスを自分で定義して演算をするのではなく、OpenCV の機能を用いて行列計算をすることが必要とされている。すなわち OpenCV の関数の扱いになれるという課題である。

1 問目は行列を定義し、その行列とその逆行列の和を計算する。まず cv::Mat の double のテンプレートサブクラスを 3×3 で定義して、値を入力して初期化した。その後、メンバ関数の t()で逆行列 At を求め、最後に出力として A+At を出力した。これで完了となる。

2 問目は行列の行列式を計算する。行列の定義は 1 問目と同様に行い、cv クラスのメンバ関数の determinant 関数を用いて行列式を計算する。これを出力して完了となる。

3 問目は行列の逆行列を求める。行列の定義を先ほどと同様に、今度は 2×2 で行い、定義した行列クラスのメンバ関数の inv()により逆行列を求めた。これを出力することで完了となる。

・実行結果

```
e2412747@a14:~/project/cpj2026_2412747_5$ ./2412747_kadai5-1
A+A^t = [2, 0, 11;
 0, 4, 8;
 11, 8, 6]

[3, 2, 1;
 1, 3, 2;
 2, 1, 3]
の行列式の演算結果は、18です。
[1, 2;
 2, 3]
の逆行列は、
[-3, 2;
 2, -1]
です。
```

図1　1問目の実行結果

2 問目

・コンパイルおよび実行方法

「g++ 2412747_kadai5-2.cpp -o 2412747_kadai5-2 $(pkg-config --cflags --libs opencv4)」とコンパイルし、「./2412747_kadai5-2 binary2026.bmp x」というように、第 1 引数にグレースケール処理をする画像のパス名、第 2 引数にグレースケール処理で用いる閾値を指定して実行する。

・実装概要

この問題は第 3 回の 4 問目で行なったグレースケール処理を OpenCV で行うというものであり、これも先ほどと同様に OpenCV の使い方になれるというものである。

まずはソース画像の Mat クラスをインスタンス化し、cv のメンバ関数である imread 関数により第 1 引数のファイルの読み取りを行なった。次に第 2 引数の閾値を stoi により int 型に変換した。先にグレースケール化を cv クラスの cvtColor メンバ関数により行い、その次に閾値によって cv クラスの threshold メンバ関数で 2 値化を行なった。ここでは閾値よりも大きい画素値のものは白色に、閾値以下のものは黒色に変換させた。そしてウィンドウ生成を namedWindow で行い、imshow で画像の表示を行うことで、出力結果の確認を容易にさせた。最後に result.bmp という名前で imwrite により画像の保存を行い、waitKey により任意のキー入力がされるまでプログラムの実行を行うよ

うにした。なおエラーは引数チェックとファイルチェック、閾値チェックで行なった。

・実行結果

閾値を 0 から 20 ずつ変化させていって 240 まで、そして 255 にした結果の出力画像を図 2-1 から 2-4 に載せた。なお左上、右上、左下、右下の順で載せている。

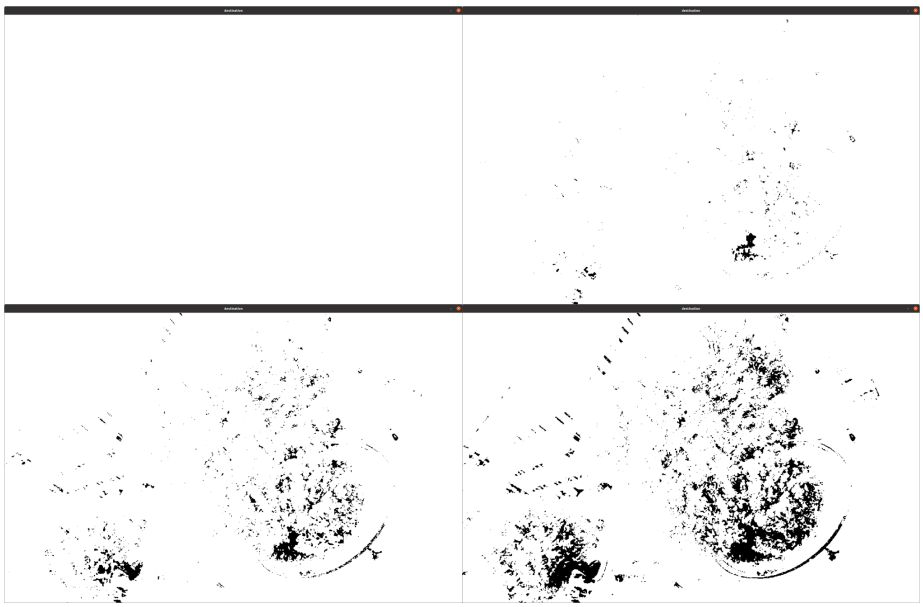

図2-1　2値化画像の出力結果(閾値0, 20, 40, 60)

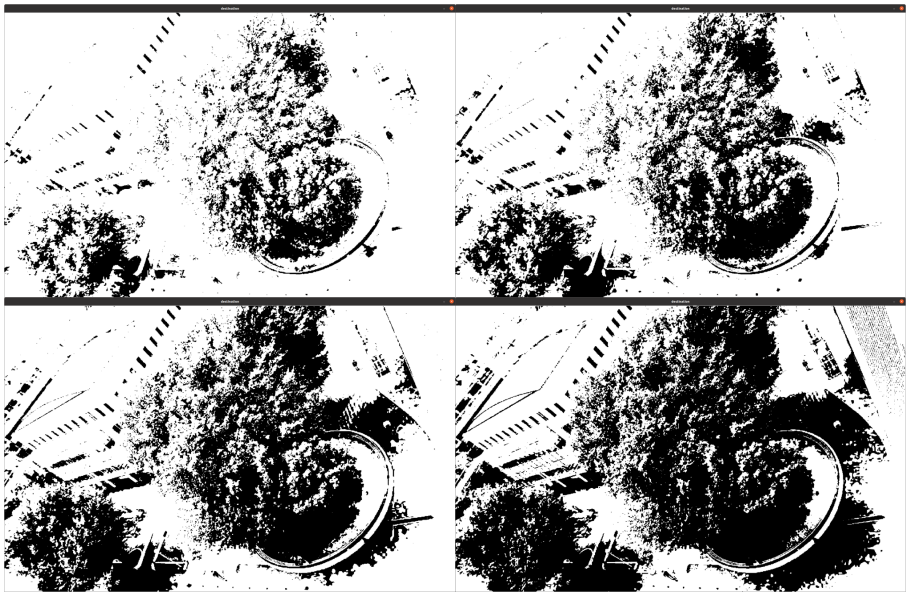

図2-2　2値化画像の出力結果(閾値80, 100, 120, 140)

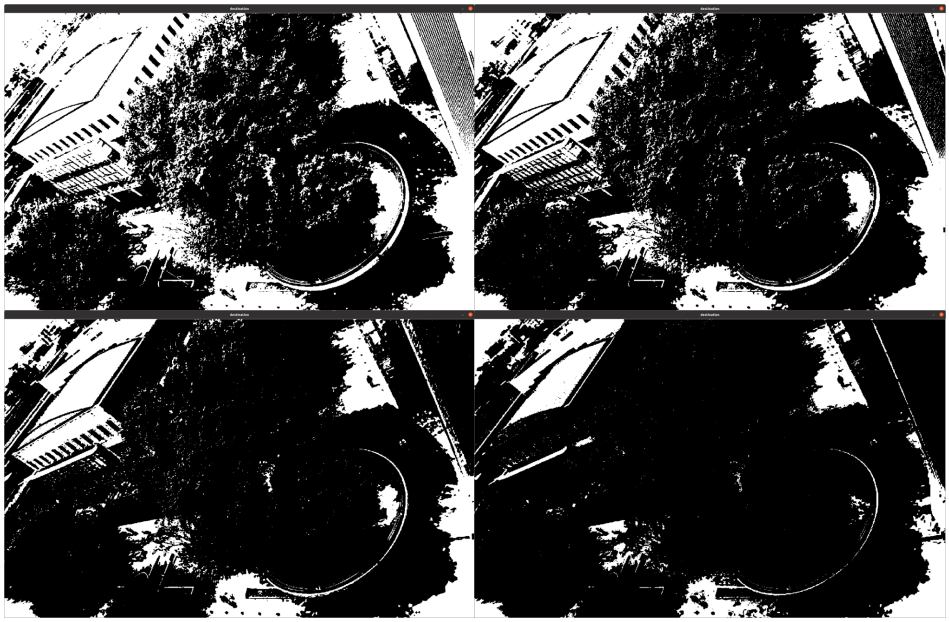

図2-3　2値化画像の出力結果(閾値160, 180, 200, 220)

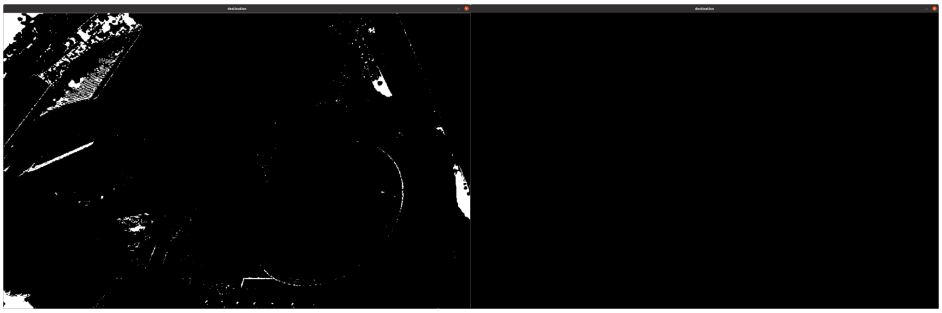

図2-4　2値化画像の出力結果(閾値240, 255)

この出力結果より、閾値を大きくしていくことで画像が全体的に白色から黒色へと変化していくことがわかる。閾値が 140 あたりが一番画像に写っているものが鮮明に写るということもわかる。

3 問目

・コンパイルおよび実行方法

「g++ 2412747_kadai5-3.cpp -o 2412747_kadai5-3 $(pkg-config --cflags --libs opencv4)」とコンパイルし、「./2412747_kadai5-3 class1tr_2026.csv class2tr_2026.csv class1te_2026.csv class2te_2026.csv」というように、第 1 引数と第 2 引数にはクラス 1 と 2 の学習データ csv ファイルのパス名を、第 3 引数と第 4 引数にはクラス 1 と 2 のテス

トデータ csv ファイルのパス名を指定して実行する。

・実装概要

この問題は OpenCV を用いて学習データより作成した識別器の SVM より、テストデータの一致率評価をするというものであり、これも OpenCV の使用になれるという側面が強く、そう難しくはない。コードは学習データの処理とテストデータの処理の 2 つに分けることができる。まずは学習データの処理について説明する。

最初に第 1 引数および第 2 引数に与えられたファイルを loadFromCSV 関数によって学習データのクラスを作成する。読み込む CSV ファイルにはヘッダがなく、全列を特徴量として利用し、ラベル列が存在しないので、それぞれ loadFromCSV の引数を 0、-2、0 と指定した。次に getSample によって CSV の特徴量のみを抽出し、vconcat で 2 クラスの特徴量データを縦方向に結合して 1 つの行列にまとめた。ただまとめたことによりどのデータがどのクラスかを SVM が判別出来なくなるので、ラベルを作成し、setTo でクラス 1 には 1 を、クラス 2 には 2 をつけた。

次に SVM の作成を行なった。今回は学習アルゴリズムを線形カーネルによる SVM にするように指定されているので、SVM パラメータ設定時に setKernel では LINEAR を選択した。SVM タイプには C_SVC、終了条件では最大反復回数と最大回数と誤差閾値を設定し、C パラメータを 1.0 に設定した。最後に SVM では float 型を必要とするため convertTo で CV_32F により float を設定、そして create および train によって実際に学習データの生成と学習を行なった。これで学習データ処理が完了した。

次にテストデータの処理について、先ほどの学習データと同様にファイルの読み取り、特徴量の抽出、ラベルの作成をまず行なった。その後、predict により SVM を用いてテストデータの分類を行なった。予測結果は pred に格納されることとなる。そしてその pred の内容と実際の内容を比較して識別率を判定した。今回はクラス 1 を正しく判定したら True Positive(TP)、クラス 2 を正しく判定したら True Negative(TN)、クラス 2 なのにクラス 1 と誤判定したら False Positive(FP)、クラス 1 なのにクラス 2 と誤判定したら False Negative(FN)をそれぞれインクリメントし、最終的に TP と TN の合計を全サンプル数から割ることで識別率を出した。そしてこの出力結果を TP、TN、FP、FN の行列形式で出力し、識別率も出力を行なった。これでプログラムが完了となる。

・実行結果

```
e2412747@a74:~/project/cpj2026_2412747_5$ ./2412747_kadai5-3 class1tr_2026.csv c
lass2tr_2026.csv class1te_2026.csv class2te_2026.csv
テストデータ数: 100
正解数: 100
正解率: 100%

                予測:クラス1    予測:クラス2
実際:クラス1          50             0
実際:クラス2          0              50
```

図3　問題3の実行結果

・考察

線形カーネルによる SVM の分類の結果、テストデータの識別率は 100%となった。線形カーネルは他のカーネルよりも計算コストが低いため、100%の精度が得られた背景にはあらかじめクラス 1 と 2 の特徴量が明確に異なっているものであったということや、入力データが完全に直線に分離可能であるということであったと考えられる。

4 問目

・コンパイルおよび実行方法

「g++ 2412747_kadai5-4.cpp -o 2412747_kadai5-4 $(pkg-config --cflags --libs opencv4)」とコンパイルし、「./2412747_kadai5-4 detection2026/」というように、引数に良性アプリケーションおよびマルウェアのデータファイル群が入ったディレクトリのパス名を指定して実行する。

・実装概要

この課題は良性アプリケーションとマルウェアの特徴量データを用いて SVM 学習をして、未知のデータが良性かマルウェアかを判別する識別器を実装するというものである。先ほどの 3 問目のコードにより拡張性を持たせた構成となっているが、全体的な流れは学習データの読み込みと分析→SVM の作成→テストデータの読み取り→識別器による分析と変わらない。

(1)と(2)では SVM に指定するカーネルが「線形」もしくは「RBF か多項式」という点、パラメータチューニングを行うかどうかという点においてのみ異なっている。(2)では出力結果を示すことが明確に指定されているが、それはどのみち(1)でも行う必要があるのでそれほど大差ない。従ってこれら以外の部分については共通であると言えるので、まずは共通部分についての解説を行う。

まずは学習 CSV ファイルの読み込みを行なった。今回はコマンドライン引数に学習データおよびテストデータの入ったディレクトリを渡すので、ディレクトリ名とそれぞれのファイル名を変数に格納し、それらを利用して loadFromCSV により CSV の読み込みを行なった。その後は getSamples による特徴量の切り出し、vconcat による統合、labels_tr や rowRange、setTo によるラベルの作成を行なった。今回は良性を 1 にマルウェアを-1 にした。そして作成したものを学習データ型として定義した。次にテスト CSV ファイルの読み込みを行なった。これも学習データと同様に行い、1つのテストデータとして作成を行なった。

機械学習の下準備が完了したので、ここから(1)線形カーネル、(2)RBF カーネルと多項式カーネルの 3 パターンそれぞれによる SVM を用いて識別器を作成した。ここでは共通で評価関数の evaluate と 3 問目と同様の TP、TN、FP、FN の混同行列での出力を行う printConfusionMatrix 関数を使用している。まずは評価関数だが、今回は構造体として評価に使用する TP、TN、FP、FN や合計データ数の total、正解数の correct、正解率の accuracy を 1 つにまとめて使用した。あまり多くの機能がない上、main 関数でも構造体のメンバにアクセスするため、クラスではなく構造体でも良いだろうと判断した。この評価関数では引数に SVM による予測結果と実際のラベルを渡し、それぞれの正誤を判断して TP、TN、FP、FN のインクリメントを行い、正答率の計算を行っている。アプリケーションごとに判定というのは for 文を用いて pred の 1 行 1 行にアクセスしていることからわかる。この結果を評価関数が構造体として返し、それを printConfusionMatrix に渡すことで結果を出力することができる。なお出力を整えるためにタブや改行を使用して何回も出力しながら調整していった。

共通項について解説が完了したので、次は個別の事項について説明する。最初に(1)で指示されている線形カーネルの SVM であるが、これは 3 問目と同様の処理を行っているだけなので特段新しいことはしていない。SVM の作成→学習データを用いた機械学習→テストデータの予測→結果の出力の手順を踏んで、線形カーネルは完了となる。

次は RBF カーネルについてである。RBF カーネルと多項式カーネルではパラメータ

の調整を自分で行い、識別率がどのようになるかを調べる必要がある。というのも線形カーネルが直線もしくは平面でしかクラスを分離できないのに対して、RBF カーネルや多項式カーネルでは特徴空間を高次元に写像し、非線形な識別境界面を実装する。OpenCV の RBF カーネルではγの値を変動させることにより過学習のしやすさを変動させることができる。また、これは他のカーネルでも同様だが、C 値を変動させることで誤分類の許容度合いを変えることができる。この 2 値を変化させて識別を行う。今回は C 値を 0.01, 0.1, 1.0, 10.0, 100.0 に、ガンマ値を 0.001, 0.01, 0.1, 1.0, 10.0 にしてそれぞれのパターンで識別を行った。それぞれの値パターンについて、SVM の作成→学習→テストデータの予測→正解率の計算を行った。そしてその中で最良のものを保存し、最後にその結果を出力することとした。

最後に多項式カーネルについてだが、こちらはガンマ値の代わりに degree 値を変動させる。C 値を 0.01, 0.1, 1.0, 10.0, 100.0、degree 値を 1.0, 2.0, 3.0, 4.0, 5.0 と選択して識別を行った。一連の流れは RBF カーネルと同じであるので割愛する。以上より、線形カーネルの SVM、そして RBF カーネルと多項式カーネルの SVM による識別が完了した。

・実行結果および識別率グラフ

実行結果は以下のようになった。

```
e2412747@a74:~/project/cpj2026_2412747_5$ ./2412747_kadai5-4 detection2026/
線形カーネル(C=1.0)
テストデータ数: 120
正解数:         83
正解率:         69.1667%

                        予測:良性       予測:マルウェア
実際:良性               46              14
実際:マルウェア         23              37

RBFカーネル パラメータチューニング結果
C \ gamma       0.001   0.010   0.100   1.000   10.000
0.010           88.333% 90.000% 91.667% 92.500% 94.167%
0.100           88.333% 90.000% 91.667% 92.500% 94.167%
1.000           88.333% 91.667% 93.333% 92.500% 93.333%
10.000          90.833% 93.333% 91.667% 92.500% 94.167%
100.000         92.500% 74.167% 42.500% 53.333% 60.000%

--- RBFカーネル 最良パラメータ ---
C=0.010, gamma=10.000
テストデータ数: 120
正解数:         113
正解率:         94.167%

                        予測:良性       予測:マルウェア
実際:良性               56              4
実際:マルウェア         3               57

多項式カーネル パラメータチューニング結果
C \ degree      1       2       3       4       5
0.01            88.33%  92.50%  76.67%  50.00%  50.00%
0.10            90.83%  30.00%  76.67%  50.00%  50.00%
1.00            69.17%  59.17%  76.67%  50.00%  50.00%
10.00           66.67%  59.17%  76.67%  50.00%  50.00%
100.00          66.67%  59.17%  76.67%  50.00%  50.00%

--- 多項式カーネル 最良パラメータ ---
C=0.01, degree=2
テストデータ数: 120
正解数:         111
正解率:         92.50%

                        予測:良性       予測:マルウェア
実際:良性               57              3
実際:マルウェア         6               54
```

図4-1　問題4(1)および(2)の実行結果

また、RBF カーネルと多項式カーネルの結果をグラフ化すると次のようになった。

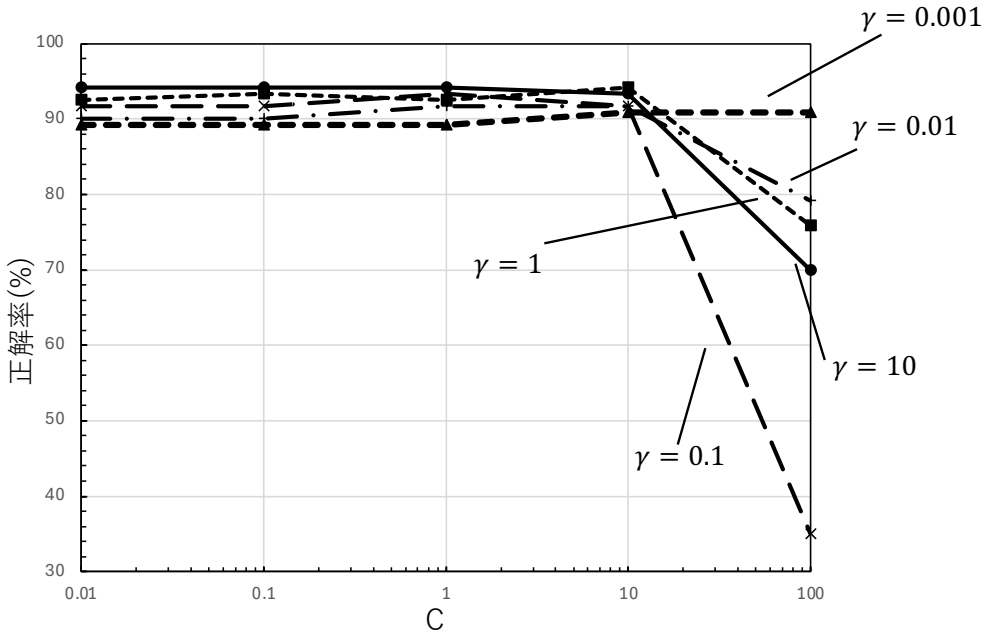

図4-2-1　RBFカーネルの実行結果のグラフ

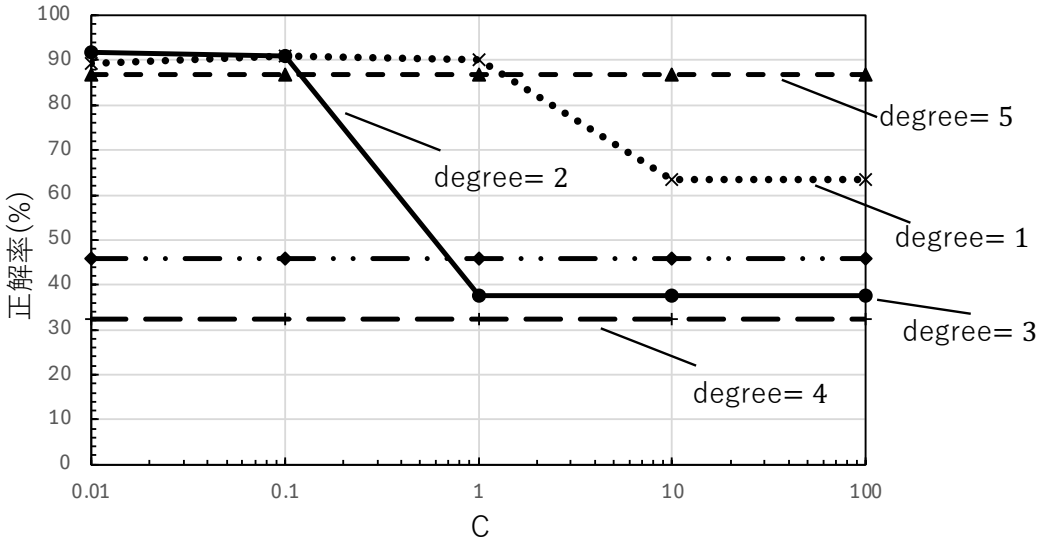

図4-2-1　多項式カーネルの実行結果のグラフ

これらより最も高い正解率を出したのは、94.2%の RBF カーネル(C=0.01、gamma=10)で、次いで 92.5%の多項式カーネル(C=0.01、degree=2)、最後に 90.0%の線形カーネルであるとわかった。RBF カーネルにおいて、ガンマは高いほど正解率が高くなるという結果が得られたが、C は大きくするとある値を超えて急激に正解率が低下することがわかった。多項式カーネルにおいて、degree の変動により正解率も変動するがそこには規則性が見えないという結果が得られ、C については RBF カーネルと同様に急激に低下する箇所があるとわかった。RBF カーネルは各パラメータの変化で正解率がそ

れぞれ変化したが、多項式カーネルではパラメータの変更に関係なく正解率が一定である区間が多数存在した。

混同行列の出力について各カーネルの特徴をまとめると、線形カーネルでは良性をマルウェアと誤検知する方がその逆よりも多かった。RBF カーネルでは両者の誤検知とも件数がほぼ一緒であり、多項式カーネルではマルウェアを良性だと誤検知する方がその逆よりも多い結果となった。

・考察

線形カーネルよりも RBF カーネルや多項式カーネルの方が高い正解率を出したということから、良性アプリケーションとマルウェアの特徴量は線形では完全に分離できない分布をしており、非線形の識別境界面の方が適していると考えられる。混同行列より誤検知について考えると、どのカーネルも誤検知はするものの線形カーネルでは安全であるものを危険だと判断している件数が多かったので、安全面では特に問題がないと推察できる。RBF カーネルは誤検知件数がほぼ等しいため、全体的な識別精度の良さが高いものであると評価が可能である。多項式カーネルは危険なものを安全であると判定している件数が多いので、これはセキュリティ面では非常に危険であり、このような識別には不向きであると結論づけることができる。

パラメータ調整について、RBF ではガンマが大きいほど各サンプルの影響範囲が狭くなり複雑な非線形境界を形成できるので、今回のデータに適合したと考えられる。しかし C が大きい場合ではガンマが大きくなると正解率が著しく悪くなるので、C とガンマの組み合わせ次第では過学習が生じていると思われる。そのためこのようなカーネルではパラメータ調整をしっかりと行うことが重要であると考えられる。多項式では次数の変化によって正解率が変化したが、次数の大小と正解率の大小の間の相関関係が見られないため、今回のようなデータには不向きであると考えられる。また C の変化が特に正解率に影響しなかったことから、degree の影響が非常に大きいということも推測できる。

以上より、本課題のようなマルウェアの特徴量の検出における識別器の作成には、RBF カーネルが一番向いているものであると言える。

5 問目

・コンパイルおよび実行方法

「g++ 2412747_kadai5-5.cpp -o 2412747_kadai5-5 $(pkg-config --cflags --libs opencv4)」とコンパイルし、「./2412747_kadai5-5 mail2026/ 5」というように、第 1 引数に学習データおよびテストデータ CSV ファイルの入ったディレクトリのパス名を、第 2 引数に CSV ファイルを何行目まで読み込むかを指定する数値を入力して実行する。

・ベイズの定理

$A_1, A_2, \ldots$を標本空間Ωの分割で、$P(A_i) > 0, i = 1, 2, \ldots$とする。$B$が事象で$P(B) > 0$ならば、

$$P(A_i|B) = \frac{P(A_i)P(B|A_i)}{\sum_{j=1}^{\infty} P(A_j)P(B|A_j)}, \qquad i = 1, 2, \ldots \tag{1}$$

が成り立つ。

（東京図書株式会社、『理工系のための入門数理統計学演習』、下川朝有、八木文香、宮岡悦良、2024 年）

・実装概要

この課題はベイズの定理の識別器によってスパムメールかどうかを判別するプログラムを実装するというものである。このプログラムでは読み込む CSV ファイルの学習に用いる行数を指定して実行する。ベイズの定理とこの点は以前の課題とは異なる点ではあるが、やはり基本的な部分は同じであるので、大枠の部分は抽象的に説明を行う。

まずは第 1 引数に指定されたディレクトリのパス名より、読み込む CSV ファイル名を構成し、そこから学習データの読み込みと特徴量の抽出を行った。ここで学習データが何行で構成されているかが確定をするので、第 2 引数のエラーハンドリングを行った。入力値が正常であれば、そこから学習データを入力値行だけ使用するようにクローンし、学習データの再作成を行った。ベイズの定理の識別器のための計算を行ったのちに、テストデータの読み込みと特徴量の切り出し、テストデータのラベル作成と統合を行い、予測と出力をして終了となった。これが全体の大枠となる。

ベイズの定理の識別器について説明を行う。今回使用する学習アルゴリズムは特徴量が正規分布に従うという仮定の下、識別を行う。そのため特徴量の平均および分散

が必要となってくる。それを computeMeanVar 関数にて行っている。分散の計算に平均を利用する関係上、先に特徴量を抽出して平均を計算し、その後に分散を計算するという構造になっている。さて、この学習アルゴリズムは字の通り式(1)で示したベイズの定理を用いる。左辺が求める確率(事後確率という)であり、右辺がその計算式となっている。これはデータ全体の確率を、学習データ数よりあらかじめ予測された事前確率と尤度の積で割るという内容である。ここで今回は対数の和を用いて計算しているが、これは小数の積同士だとプログラム内でアンダーフローが発生し、正確な値を計算することができなくなるという問題が発生するため、それを防ぐために行っている。このようにして識別を行うという仕組みになっている。

実際の計算だが、まずは computeMeanVar によって学習データの特徴量の平均と分散を求めた。学習データ数は両者等しいため、事前確率は通常とスパムで同一とした。そしてこれらを bayesPredict 関数の引数として渡すことで計算を行った。この関数では、まず対数の事前確率を設定し、次に対数の尤度を計算している。尤度の計算は logGaussian 関数で行っており、これは正規分布での計算方法にならって計算を行っている。対数の事後確率は対数の事前確率と対数の尤度の和を対数の全体の確率で引くことによって求めると先に述べたが、事後確率が通常とスパムのどちらが大きかを判別して識別するため、全体の確率というものは定数として扱うことができ、排除できるため考えなくて良いこととなる。したがって事前確率と尤度によって求まる値の比較によって通常とスパムの判定を行った。そして今までと同様に混同行列の判定を行い、正解率を算出した。これでベイズの定理の識別器の説明が完了した。最後に識別の結果を出力してこのプログラムは完了となる。

・実行結果

実行結果をそれぞれ以下の図に載せた。

```
e2412747@a74:~/project/cpj2026_2412747_5$ ./2412747_kadai5-5 mail2026/ 5
学習データ数: 5 行
テストデータ数: 200
正解数:         148
正解率:         74%

                        予測:良性       予測:マルウェア
実際:良性               49              51
実際:マルウェア         1               99
e2412747@a74:~/project/cpj2026_2412747_5$ ./2412747_kadai5-5 mail2026/ 10
学習データ数: 10 行
テストデータ数: 200
正解数:         163
正解率:         81.5%

                        予測:良性       予測:マルウェア
実際:良性               65              35
実際:マルウェア         2               98
e2412747@a74:~/project/cpj2026_2412747_5$ ./2412747_kadai5-5 mail2026/ 50
学習データ数: 50 行
テストデータ数: 200
正解数:         191
正解率:         95.5%

                        予測:良性       予測:マルウェア
実際:良性               95              5
実際:マルウェア         4               96
e2412747@a74:~/project/cpj2026_2412747_5$ ./2412747_kadai5-5 mail2026/ 100
学習データ数: 100 行
テストデータ数: 200
正解数:         192
正解率:         96%

                        予測:良性       予測:マルウェア
実際:良性               95              5
実際:マルウェア         3               97
e2412747@a74:~/project/cpj2026_2412747_5$ ./2412747_kadai5-5 mail2026/ 200
学習データ数: 200 行
テストデータ数: 200
正解数:         193
正解率:         96.5%

                        予測:良性       予測:マルウェア
実際:良性               97              3
実際:マルウェア         4               96
```

図5　問題5での5~200行までの実行結果

図より 5 行、10 行、50 行、100 行、200 行の正解率はそれぞれ 74%、81.5%、95.5%、96%、96.5%となったことがわかる。これより学習データの増加に伴って正解率が単調上昇していったことが確認できる。

・考察

学習データが 5 行や 10 行など極端に少ない場合では正解率が低かったが、これはサンプル数が少ないことによる推定された平均や分散の偏りが生じ、真の値からズレることによって発生したと考えられる。学習データ数が 50 行になると途端に正解率が上昇し、以後 100 行や 200 行のときと大差ないほどに精度が良かったが、これは今回のデータを判別するには学習データ量が 50 ほど、すなわち全体の 25%あれば十分であったということを指していると捉えられる。統計的推定の観点から考えると、サンプル数の増加に対し標準誤差は$\sqrt{1/n}$に比例して小さくなっていくので、サンプル数の増加の過程でこのような結果になったと推測できる。学習データ量が 100 行や 200 行のときは正解率にほとんど変化が見られなかったが、これはベイズの定理の識別器が、特徴量が正規分布にしたがっているとして演算を行ったことによるものだと考えられ、100%正しく判別できないのはこの学習アルゴリズムの識別における限界が存在することを示していると思われる。ただ、今までの問題で使用した線形カーネルや RBF カーネル、多項式カーネルに比べて非常に良い精度を出しているので、この点が劣っているということは決してないと結論づけることができる。

6 問目

・コンパイルおよび実行方法

「g++ 2412747_kadai5-6.cpp -o 2412747_kadai5-6 $(pkg-config --cflags --libs opencv4)」とコンパイルし、「./2412747_kadai5-6 ball2026/」というように、引数に unknown 画像が入ったディレクトリのパス名を指定して実行する。

・実装概要

この課題は、未知画像が学習データとして与えられた画像のどれであるかを判別するプログラムを作成するという課題であり、今までの SVM の実装を発展させた内容となっている。今までは特徴量が与えられ、SVM のカーネルも指定されていたが、今回は自分で決めなければならない。そこで、カラー画像が与えられるという点や各ボールの画像には色だけでなく彩度や明暗が関わってくるという点、サンプル数がたったの 9 つしかないという点から考え、特徴量として BGR ではなく HSV を使用し、学習アルゴリ

ズムはこれまで課題から RBF カーネルが最良であると判断したためそれを利用した。コードがどのような仕組みなっているかを順番に解説していく。

まずは引数に指定したディレクトリ名と ball2026 に入っているファイル名よりエントリの構成を行った。ここで各画像に対して野球が 1、バスケが 2、サッカーが 3 というラベリングを行ったのだが、ファイル名とラベルを別で持つのが面倒である上、ファイル名の変数を 9 つも定義するのが冗長であると感じたため、vector の pair を使用して train_files を定義した。

次に学習データ行列と正解用ラベル行列を定義した。今回は HSV ヒストグラムを用いたため、H、S、V が各 8 次元で合わせて 24 次元であるので、24 次元の特徴量を持つ行列として定義した。そして先ほど用意した train_files からそれぞれ画像の読み込みおよびデータの格納とラベルの格納を行った。これで学習データの用意ができた。

今回は機械学習アルゴリズムとして RBF カーネルを選んだが、課題 4 で実験したように C 値とガンマ値の選定が重要となる。そのためそれぞれ任意の 5 つの値、合計 25 パターンを選択し、それぞれで機械学習を行った。しかしどの値が最良であるのかというものは実際にテストを行わないとわからないものである。そこでサンプルデータ数が 9 個と極めて少ないという点と合わせて、1 つ抜き交差検証(LOOCV)と呼ばれる方法を利用することにした。これはサンプルデータのうち 1 つをテストデータ、他すべてを学習データとして利用しモデルを作成し、全パターンで検証を行うことで精度の評価を行うことができる手法である。これを全パラメータ組み合わせについて for 文を利用して行った。1 回ごとに SVM の作成を行うため、今回のようにサンプルが少ない場合であれば良いが、多くなるとこの手法は不向きである。LOOCV により最良のパラメータを判別し、そのパラメータを用いて実際の画像識別用の RBF カーネルの SVM による識別器を作成することにした。

これまでの工程によって未知画像を識別する準備は完了したので、これより識別を行うこととした。しかし機械学習では SVM は画像そのものを理解することができないので、SVM が理解できるものへと変換しなければならない。そのため imread で読み取った img 行列を引数として extractFeature という自作関数を実行することによって、画像データを特徴量行列へ変換することにした。この関数ではまず画像データを BGR から HSV へと変換している。OpenCV は通常、画像を BGR として保持し、今回は先述したように画像の特徴から色相、彩度、明度を成分として持つ HSV に変換することにしたので、この変換を行った。そして HSV へ変換したら、各成分について vector を用いて分離を行い、各 8 次元で計 24 次元の特徴量行列を 0 初期化しつつ定義した。次に

H と S と V それぞれのヒストグラムの計算を行った。ここでは全部のピクセルを走査し、各ピクセルの H、S、V 値がどの分布の部分に収まるかを計算している。H 値は 0～179、S 値と V 値は 0～255 が範囲であるので、それを 8 等分して判断した。この判定により feature 行列の対応するインデックスをインクリメントし、特徴量行列の形成を行った。全ての判定が終了したのち、この特徴量を総ピクセル数で割って正規化した。これは画像がいかなるサイズであっても対応できるようにするための処置であるのに加えて、特徴量の把握がしやすくなるというメリットがある。

これによりテストデータを SVM が識別できるようになったので、後は SVM による予測を行い、その識別結果を出力した。出力には labelToName 関数を利用し、これによりラベルとボールの種類を対応させることにした。以上より全工程が終了となる。

・実行結果

```
e2412747@a74:~/project/cpj2026_2412747_5$ ./2412747_kadai5-6 ball2026/
C \ gamma        0.00     0.01     0.10     1.00     10.00
0.10             0.00%    0.00%    0.00%    0.00%    0.00%
1.00             0.00%    0.00%    0.00%    33.33%   66.67%
10.00            0.00%    0.00%    33.33%   77.78%   88.89%
100.00           0.00%    33.33%   77.78%   100.00%  88.89%
1000.00          33.33%   77.78%   100.00%  100.00%  88.89%

最良パラメータ: C=100.00, gamma=1.00

unknown画像の識別結果
ball2026/unknown1.ppm
        -> 識別結果: バスケットボール
ball2026/unknown2.ppm
        -> 識別結果: サッカーボール
ball2026/unknown3.ppm
        -> 識別結果: 野球ボール
```

図6　問題6の実行結果

・考察

実行より 3 枚の未知画像がそれぞれバスケットボール、サッカーボール、野球ボールであったと正しく判定できたが、これにはいくつかの理由が関与していると考えられる。1 つ目はパラメータ値の選択および LOOCV である。この課題では少ないデータをいかにして運用するかが問われていたと感じたので LOOCV を利用したが、これとパラメータチューニングを行うことで識別の幅を広げることができたと考えられる。2 つ目は学習およびテストデータの画像が 3 種類ともはっきりと区別することができたという

ことである。バスケットボールがオレンジ色なのに対してサッカーボールと野球ボールはメインが白色であるためこの点で区別ができ、サッカーボールは白と黒で構成されるので野球ボールと比べて明度が低い範囲にも分布することになる。このようにして 3 種類の特徴が明確に分かれていることが理由として考えられる。ただ逆を言えば、学習データの画像が、背景が大部分を占めているや、赤とピンク、青と青紫など似ているなど酷似している場合では、この HSV ヒストグラムによる識別は高い精度を出せないと推測できる。